Level 2: Aggregate Functions

Aggregate functions collapse many rows into a single summary number: COUNT, AVG, MAX, MIN, and SUM. Paired with GROUP BY, they let you compute per-category summaries (average salary per department, headcount per team) instead of one grand total, and HAVING lets you filter those group results.

Q26. Total employees EASY

```
SELECT COUNT(*) AS total_employees FROM employees;
```

WHAT THIS QUERY DOES

`COUNT(*)` is your first **aggregate function** — instead of returning many rows, it collapses the whole table down into a single number: how many rows exist. The `AS total_employees` part just renames the output column to something more readable than the default `COUNT(*)` label.

RESULT ON OUR SAMPLE DATA

total_employees
15

1 row returned.

Q27. Average salary EASY

```
SELECT AVG(salary) AS avg_salary FROM employees;
```

WHAT THIS QUERY DOES

`AVG()` adds up every value in the column and divides by the number of rows — the familiar 'mean' from school math, computed across all 15 employees' salaries in one step.

RESULT ON OUR SAMPLE DATA

avg_salary
75,333.33

1 row returned.

Q28. Maximum salary EASY

```
SELECT MAX(salary) AS max_salary FROM employees;
```

WHAT THIS QUERY DOES

`MAX()` scans the whole column and returns the single largest value. Unlike Q23 (`ORDER BY ... LIMIT 1`), this gives you *just the number*, not the full row it came from — often all you need, and slightly cheaper for the database to compute.

RESULT ON OUR SAMPLE DATA

max_salary
95,000

1 row returned.

Q29. Minimum salary EASY

```
SELECT MIN(salary) AS min_salary FROM employees;
```

WHAT THIS QUERY DOES

The mirror image of `MAX()` — `MIN()` returns the smallest value in the column.

RESULT ON OUR SAMPLE DATA

min_salary
58,000

1 row returned.

Q30. Salary sum EASY

```
SELECT SUM(salary) AS total_salary FROM employees;
```

WHAT THIS QUERY DOES

`SUM()` adds every value in the column together. This is exactly what a company's total monthly payroll would be, computed straight from the raw per-employee data.

RESULT ON OUR SAMPLE DATA

total_salary
1,130,000

1 row returned.

Q31. Count employees per department EASY

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

This is where aggregates get powerful: `GROUP BY department` splits the table into buckets — one bucket per distinct department — and `COUNT(*)` is then computed *separately for each bucket* instead of the whole table at once. The result is one row per department, each with its own headcount.

RESULT ON OUR SAMPLE DATA

department	employee_count
Finance	3
HR	3
IT	7
Marketing	2

4 rows returned.

Q32. Average salary per department EASY

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Same grouping idea as Q31, just swapping which aggregate runs inside each department bucket — here it's the average salary *within* that department, not a headcount.

RESULT ON OUR SAMPLE DATA

department	avg_salary
Finance	74,000
HR	63,000
IT	84,714.29
Marketing	63,000

4 rows returned.

Q33. Highest salary per department EASY

```
SELECT department, MAX(salary) AS highest_salary
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Again, same `GROUP BY` skeleton, this time with `MAX()` — the top earner's salary computed independently for each department, side by side in one result set instead of needing four separate queries.

RESULT ON OUR SAMPLE DATA

department	highest_salary
Finance	85,000
HR	65,000
IT	95,000
Marketing	68,000

4 rows returned.

Q34. Lowest salary per department EASY

```
SELECT department, MIN(salary) AS lowest_salary
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

The `MIN()` version of the same pattern — the lowest-paid salary within each department.

RESULT ON OUR SAMPLE DATA

department	lowest_salary
Finance	67,000
HR	60,000
IT	72,000
Marketing	58,000

4 rows returned.

Q35. Departments with more than 5 employees MEDIUM

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

WHAT THIS QUERY DOES

This introduces `HAVING` — a filter that runs *after* grouping, on the aggregated results. It's easy to confuse with `WHERE`, but they operate at different stages: `WHERE` filters individual rows *before* grouping happens, while `HAVING` filters entire groups *after* their aggregate (here, the count) has been computed. You can't write `WHERE COUNT(*) > 5` because at the point `WHERE` runs, no counting has happened yet.

RESULT ON OUR SAMPLE DATA

department	emp_count
IT	7

1 row returned.

Q36. Average age EASY

```
SELECT AVG(age) AS avg_age FROM employees;
```

WHAT THIS QUERY DOES

Same `AVG()` idea as Q27, applied to the age column instead of `salary` — aggregate functions work the same way regardless of which numeric column you point them at.

RESULT ON OUR SAMPLE DATA

avg_age
36.40

1 row returned.

Q37. Maximum experience EASY

```
SELECT MAX(experience_years) FROM employees;
```

WHAT THIS QUERY DOES

Notice this query has no AS alias — that's perfectly valid SQL, it just means the output column will be labeled with the raw expression (`MAX(experience_years)`) instead of a friendly name. Aliases are a readability choice, not a requirement.

RESULT ON OUR SAMPLE DATA

MAX(experience_years)
10

1 row returned.

Q38. Salary greater than average MEDIUM

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

WHAT THIS QUERY DOES

A subquery inside a `WHERE` clause: the inner query computes one single number (the average salary, ≈75,333), and the outer query then filters individual employees against that number. This is a common way to answer 'who is above/below the average' — the average itself doesn't need to be a separate step you run manually first.

RESULT ON OUR SAMPLE DATA

name	salary
Rahul Singh	85,000
Vikram Reddy	90,000
Karthik Nair	95,000
Arjun Reddy	82,000
Suresh Babu	88,000
Ravi Teja	91,000

6 rows returned.

Q39. Employees earning minimum salary MEDIUM

```
SELECT name, salary FROM employees
WHERE salary = (SELECT MIN(salary) FROM employees);
```

WHAT THIS QUERY DOES

Same subquery pattern as Q38, but matching *equal to* the minimum rather than *greater than* the average. Note this correctly returns every employee tied for the

lowest salary, not just one — unlike `ORDER BY salary LIMIT 1` (Q24), which would only ever show a single row even if there were a tie.

RESULT ON OUR SAMPLE DATA

name	salary
Sneha Patel	58,000

1 row returned.

Q40. Total bonus EASY

```
SELECT SUM(bonus) AS total_bonus FROM employees;
```

WHAT THIS QUERY DOES

Another straightforward `SUM()`, this time totalling the `bonus` column across every employee.

RESULT ON OUR SAMPLE DATA

total_bonus
151,500

1 row returned.

Q41. Average bonus EASY

```
SELECT AVG(bonus) AS avg_bonus FROM employees;
```

WHAT THIS QUERY DOES

The average bonus paid out per employee, computed the same way as the average salary in Q27.

RESULT ON OUR SAMPLE DATA

avg_bonus
10,100

1 row returned.

Q42. Count NULL values MEDIUM

```
SELECT COUNT(*) - COUNT(manager_id) AS null_managers FROM employees;
```

WHAT THIS QUERY DOES

A subtle but important detail: `COUNT(*)` counts *every row*, but `COUNT(column_name)` only counts rows where that specific column is **not NULL** — NULLs are silently skipped. Subtracting the two tells you how many rows have a NULL in that column, here revealing there's exactly 1 employee with no manager (matching what Q10 found directly).

RESULT ON OUR SAMPLE DATA

null_managers
1

1 row returned.

Q43. Count non-NULL values EASY

```
SELECT COUNT(manager_id) AS non_null_managers FROM employees;
```

WHAT THIS QUERY DOES

As explained in Q42, `COUNT(column)` automatically ignores NULLs — so this simply counts how many employees *do* have a manager assigned, without needing an explicit `WHERE manager_id IS NOT NULL` clause.

RESULT ON OUR SAMPLE DATA

non_null_managers
14

1 row returned.

Q44. Department with highest average salary MEDIUM

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC LIMIT 1;
```

WHAT THIS QUERY DOES

This chains three concepts together: group by department, compute each group's average salary, then apply the familiar 'sort then take the top 1' trick from Q23 — but now sorting the *grouped/aggregated* results rather than raw rows.

RESULT ON OUR SAMPLE DATA

department	avg_salary
IT	84,714.29

1 row returned.

Q45. Department with lowest average salary MEDIUM

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary ASC LIMIT 1;
```

WHAT THIS QUERY DOES

Identical to Q44 with the sort direction flipped — ascending order surfaces the department with the *lowest* average salary at the top instead.

RESULT ON OUR SAMPLE DATA

department	avg_salary
HR	63,000

1 row returned.

Q46. Highest bonus EASY

```
SELECT MAX(bonus) FROM employees;
```

WHAT THIS QUERY DOES

Straightforward `MAX()` on the bonus column, same pattern as Q28 applied elsewhere.

RESULT ON OUR SAMPLE DATA

MAX(bonus)
20,000

1 row returned.

Q47. Lowest bonus EASY

```
SELECT MIN(bonus) FROM employees;
```

WHAT THIS QUERY DOES

Straightforward `MIN()` on the bonus column.

RESULT ON OUR SAMPLE DATA

MIN(bonus)
4,000

1 row returned.

Q48. Average experience EASY

```
SELECT AVG(experience_years) FROM employees;
```

WHAT THIS QUERY DOES

The average number of years of experience across all employees — same `AVG()` pattern, different column.

RESULT ON OUR SAMPLE DATA

AVG(experience_years)
5.33

1 row returned.

Q49. Total projects EASY

```
SELECT COUNT(*) FROM projects;
```

WHAT THIS QUERY DOES

`COUNT(*)` isn't limited to the `employees` table — it works identically on any table. This counts how many rows exist in `projects`.

RESULT ON OUR SAMPLE DATA

COUNT(*)
6

1 row returned.

Q50. Total managers MEDIUM

```
SELECT COUNT(DISTINCT manager_id) FROM employees
WHERE manager_id IS NOT NULL;
```

WHAT THIS QUERY DOES

Combines three ideas from earlier: filter out NULLs first (`WHERE manager_id IS NOT NULL`, as in Q19), narrow to unique values (`DISTINCT`), then count them. The result is the number of *distinct people* who manage at least one employee — not the number of employees who have a manager (that was Q43), but how many unique managers exist among them.

RESULT ON OUR SAMPLE DATA

COUNT(DISTINCT manager_id)
5

1 row returned.